Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**Programming from First Principles**	
Topic Name	**Block Structure 1.2**	
Topic Code	**-**	
Unit/Module No. & Name	**1**	**Standard Constructs**
Self-Learning Hours	**-**	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-11
4.0	Examples And Case Studies	12-18
5.0	Keywords And Touch Vocabulary	19-20

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. PL/SQL is a programming language that works with SQL inside an Oracle database.
2. SQL is good for single actions like selecting or updating data.
3. Many real tasks need multiple steps, decisions, and error handling.
4. PL/SQL adds these features to SQL.
5. The main unit of PL/SQL is called a block.
6. A block is a small program with a clear start and end.
7. A block runs as one complete unit.
8. A block can handle errors safely.
9. This unit teaches PL/SQL block structure.
10. It is designed for Online MCA students.
11. Students learn how to read, write, and test blocks.
12. Blocks are important for exams and real database systems.
13. Understanding blocks helps in learning procedures, functions, triggers, and packages faster.
14. Students learn the parts of a PL/SQL block.
15. Students learn the correct order of block sections.
16. Students learn where variables are declared.
17. Students learn how variables get values.
18. Students learn how SQL works inside PL/SQL.
19. Students learn that SELECT must use INTO in PL/SQL.
20. Students learn to use decisions and loops in a block.
21. Students learn to handle exceptions like NO_DATA_FOUND and ZERO_DIVIDE.
22. Students learn a simple practice method using a block skeleton.
23. After this unit, students can write a correct block skeleton.
24. Students can use proper semicolons.
25. Students can identify required and optional sections of a block.
26. Students can declare NUMBER, VARCHAR2, and DATE variables.
27. Students can assign values using the PL/SQL assignment operator (:=).
28. Students can read one row from a table using SELECT INTO.
29. Students can use values in calculations or output messages.
30. Students can write a basic IF statement.
31. Students can write a basic loop.
32. Students understand scope and variable lifetime.
33. Students know why variables disappear after a block ends.
34. Students understand why a block is called a single unit.
35. Students can format code with clear indentation.
36. Neat code is easy to debug and easy to grade.
37. Students can connect block structure to college systems.
38. Examples include fee payment checks, result processing, and library fine rules.

1.0 INTRODUCTION

1.1 Databases and SQL

A database stores important facts in tables. A college database can store student details, course registrations, marks, fee records, library issues, and login data. SQL is the standard language used to work with this data. With SQL, you can SELECT data, INSERT new rows, UPDATE existing rows, and DELETE rows. SQL is clear and powerful when a task fits into one statement.

1.2 Why SQL Alone Is Not Enough

Many real tasks are not one statement. A portal may need to check a rule, calculate a value, save a record, and then show a message. A banking system may need to verify a balance, create a transaction entry, update an account, and handle failure safely. A result system may need to read many student rows, decide a grade for each, and update the table. These tasks need ordered steps. They also need decisions, like IF a fee is unpaid THEN stop. They may need loops, like FOR each student row DO update grade.

Another reason SQL alone is not enough is that SQL has limited memory for a running task. In many problems, you need to store a value, change it, and use it again. For example, you may read a mark, add bonus points, and then compare the new total to a pass rule. You may also need a counter to count how many rows were updated. PL/SQL gives you variables for this. It also gives you loops, so you can repeat a small step many times without writing the same line again in safe ways.

1.3 How PL/SQL Helps

PL/SQL adds procedural features around SQL. “Procedural” means it supports steps, conditions, and loops. PL/SQL lets you declare variables, run SQL statements, and then use the results in later steps. It also lets you handle errors in a planned way, so the program does not stop without a clear reason. Because PL/SQL runs close to the data, it can apply business rules in a consistent way. For example, a “no negative payment” rule can be enforced every time a payment is saved.

1.4 Tools and Execution

Students often write PL/SQL in tools like SQL*Plus, Oracle SQL Developer, or online practice consoles. In these tools, each statement ends with a semicolon. A PL/SQL block ends with END; and many tools use a slash on a new line to execute the whole block. For learning, students often enable output and use DBMS_OUTPUT.PUT_LINE to print messages. This output is mainly for practice and debugging. In real applications, output is usually returned to an app screen or saved in tables, not printed. Still, DBMS_OUTPUT is very useful in a classroom. It lets you show the value of a variable at a certain step. It also lets you confirm that a loop is running the expected number of times. Many online learners use DBMS_OUTPUT like a small flashlight to see what the code is doing. When you move to projects, you may replace this with logging to a table or with messages sent to the application.

1.5 Common Beginner Errors

Beginners often mix SQL rules with PL/SQL rules. In PL/SQL, assignment uses :=, not =. Another common error is forgetting INTO in a SELECT statement. In plain SQL, a SELECT can return many rows to the screen. In PL/SQL, SELECT must put its result into variables using INTO, and it must return exactly one row. If it returns no row, NO_DATA_FOUND happens. If it returns more than one row, TOO_MANY_ROWS happens. These exceptions are common in exams and labs, so students should learn them early. Another beginner error is forgetting a semicolon at the end of a statement. A missing semicolon can stop the whole block from compiling. Some learners also forget to declare a variable before use. Others use a variable name that is the same as a column name and get confused. A safe habit is to use a prefix like v_ for variables. Another common confusion is NULL. In SQL, NULL means “unknown.” Comparing with NULL using = does not work the way many beginners expect. In PL/SQL, you often check NULL with IS NULL or IS NOT NULL to keep logic correct.

2.0 MEANING AND DEFINITION

2.1 Meaning of Block Structure

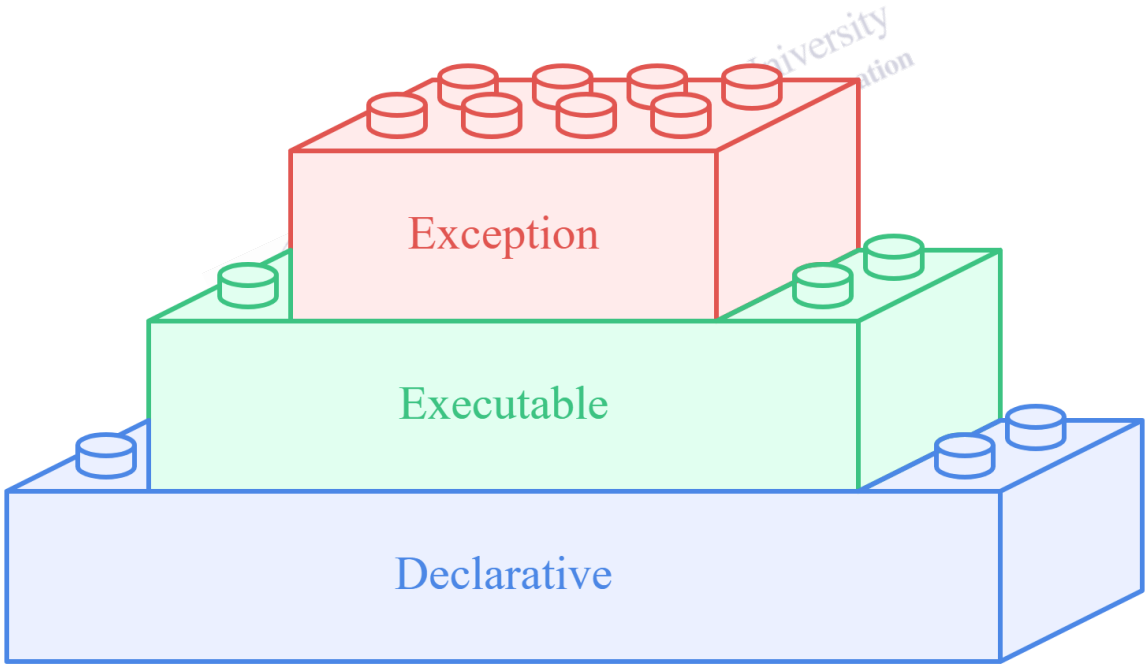
A PL/SQL block is a structured unit of code that is compiled and executed as one program unit. It is like a small workspace for one task. It has its own names, such as variables, and its own statements. It starts, runs, and ends as a unit. This boundary is important because it creates order and control. It tells the database engine, "run this whole unit together."

2.2 Parts of a Block

A standard block can have three sections, written in a fixed order: DECLARE, BEGIN, and EXCEPTION, and it always ends with END. The DECLARE section is optional. The BEGIN section is mandatory. The EXCEPTION section is optional. Even when a section is optional, the order does not change.

Fig. 1

PL/SQL Block Structure



The image shows the PL/SQL block structure with three layers: Declarative for variable definitions, Executable for main program logic, and Exception for handling runtime errors safely and systematically.

2.2.1 Declarative Part

The DECLARE part is where you list the items you will use. These items include variables, constants, cursors, records, and user-defined exceptions. A variable is a name that holds a value.

A constant is a fixed value that should not change. A cursor is a way to work with many rows, one at a time. A record can hold many fields together, like one row. Declaring names before use makes code clearer and prevents many mistakes. It also helps other people read your code.

2.2.2 Executable Part

The BEGIN part is where the work happens. It contains SQL statements and PL/SQL statements. SQL statements include SELECT, INSERT, UPDATE, and DELETE. PL/SQL statements include assignment using :=, decisions using IF, and repetition using loops. The executable part can also call stored procedures and functions. This part is often the longest part because it contains the main business logic.

2.2.3 Exception Part

The EXCEPTION part is where you handle errors. An exception handler starts with WHEN, names the exception, and lists actions to take. For example, WHEN NO_DATA_FOUND THEN you might print a message like “No record found.” A general handler, WHEN OTHERS, catches any error not caught earlier. It should be used carefully. It should not hide problems. A good handler can show SQLERRM, which is the database error message text, and it can also log the error for later review.

2.3 Variables and Data Types

PL/SQL variables have data types. Common types are NUMBER, VARCHAR2, and DATE. A NUMBER can store counts, marks, or amounts. A VARCHAR2 can store names and messages. A DATE can store dates like a payment date or a due date.

You can declare constants in the DECLARE section. A constant uses the keyword CONSTANT and a fixed value. Constants help when a value should not change during the block. For example, you can store a late fee rate as a constant. This makes the rule clear and prevents accidental changes.

You can also declare records. A record can hold many related values together. One easy way is %ROWTYPE, which matches a full table row shape. For example, v_student student%ROWTYPE can hold fields from the student table. Records reduce the number of separate variables when you need many columns.

You can also declare a variable using %TYPE, such as v_name student.student_name%TYPE. This means the variable uses the same type as the column. This is useful because it keeps code safe if the column type changes later. You can also use %ROWTYPE to hold a full row shape, which is helpful when many columns are needed.

2.4 Scope, Lifetime, and Nesting

Scope is the area where a name can be used. If you declare a variable in a block, you can use it inside that block. When the block ends, the variable is gone. That is the variable lifetime. Blocks

can be nested. An inner block can use variables from an outer block. But the outer block cannot use variables declared only in the inner block. This rule reduces confusion and helps prevent accidental use of temporary values. Scope also helps you reuse simple names safely. For example, you can use v_total in two different blocks, and they do not interfere, because each block has its own scope. This is like having two notebooks for two different tasks. Each notebook can have a page called “total,” but the pages are not the same page.

Exceptions also follow a scope rule. If an exception happens in an inner block, PL/SQL first looks for a handler in that inner block. If none matches, the exception propagates to the outer block. This behavior allows local handling for small errors while still letting large errors be handled at a higher level.

2.5 Definition in Academic Form

A PL/SQL block is the fundamental unit of PL/SQL program structure. It consists of an optional declarative section, a mandatory executable section, and an optional exception-handling section, written in that order and executed as a single unit.

3.0 NATURE OR TYPE

3.1 Structured Nature

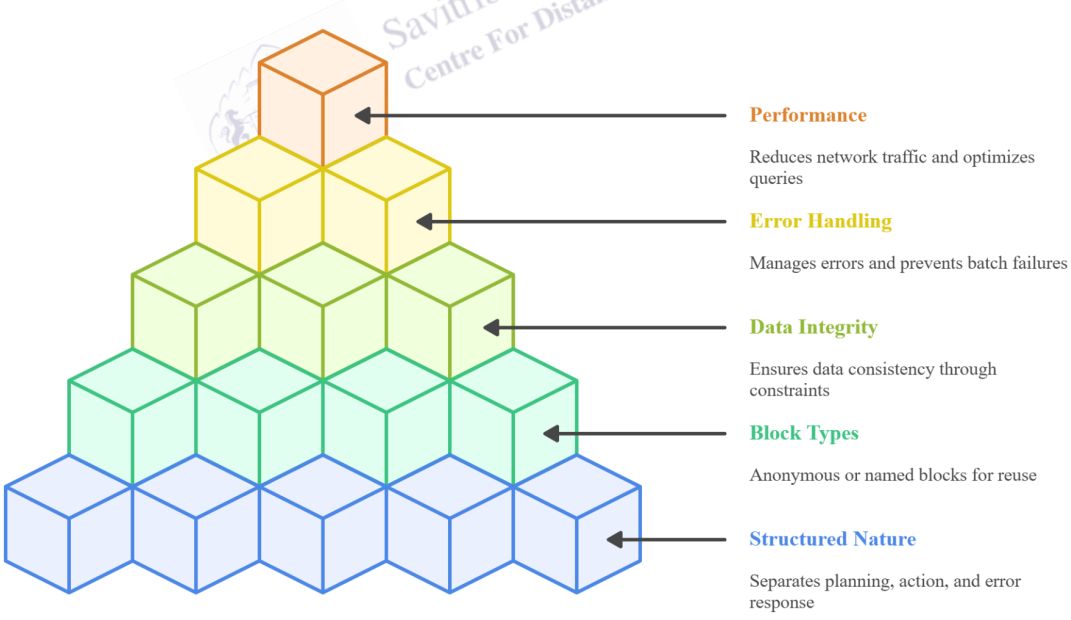
Block structure shows that PL/SQL is a disciplined language. It separates planning, action, and error response. This separation makes code easier to read. It also makes code easier to test. A reader can scan the DECLARE part to see what names exist. Then the reader can scan the BEGIN part to see the steps. Then the reader can scan the EXCEPTION part to see what happens when something goes wrong.

3.2 Anonymous and Named Blocks

Blocks can be anonymous or named. An anonymous block has no name and is not stored in the database. Students use anonymous blocks for practice and one-time scripts. A named block is stored and can be reused. Procedures and functions are named blocks. Triggers are named blocks that run automatically when an event happens, such as an INSERT on a table. Packages are named containers that group related procedures and functions. Even though these program units have extra syntax, their logic still uses blocks inside. This is why block structure appears again and again in database programming topics.

Fig. 2

PL/SQL Block Structure Hierarchy



The image explains PL/SQL block structure hierarchy, showing structured nature at the base, block types, data integrity, error handling, and performance benefits layered step by step for efficient database programming.

3.3 Integrity, Transactions, and Consistency

Databases protect data using constraints, such as primary keys, foreign keys, not null rules, and check constraints. When a block inserts or updates data, constraints may reject invalid values. A good block can catch the error and show a clear message. A good block also respects transaction safety. A transaction is a set of changes treated as one unit. If a task has many updates, the system should not be left half updated. In many applications, the application layer decides when to COMMIT or ROLLBACK. In a web app, the screen action may call a procedure, and the app may commit only after it receives a success message. This helps keep control in one place.

In scripts and batch jobs, commits may be placed inside blocks. Even then, it is wise to commit in a planned way. If you commit too often, you can slow the job. If you commit too late, you risk losing too much work if an error occurs. Some developers also use SAVEPOINT, which is like a small checkpoint inside a transaction. A savepoint lets you roll back part of the work without rolling back everything. In beginner work, you can remember one simple rule: do not leave the database half updated.

Blocks can also use locks in a safe way. In the bank transfer example, FOR UPDATE locks the row so another session cannot change the balance at the same time. This supports correctness in systems where many users act at once.

3.4 Error Propagation and Local Handling

A block can be nested inside another block. This supports local control. For example, an outer block may loop through many students. An inner block may compute a scholarship amount for one student. If the inner block fails due to missing data, it can handle the error and continue with the next student. This prevents one bad row from stopping a full batch. This is a common pattern in result processing and data cleaning.

3.5 Performance and Maintainability

Block structure also supports maintainability. Small blocks are easier to understand than one long script. Clear variable names reduce mistakes. Consistent indentation makes IF and LOOP levels easy to see. In performance terms, PL/SQL can reduce network traffic because work runs on the database server rather than sending many small SQL calls over the network. However, performance still needs care. For example, row-by-row updates can be slow for very large tables. Students should first learn correctness and clarity, and then learn performance ideas step by step. One simple performance habit is to avoid doing the same SELECT many times inside a loop if the value does not change. Another habit is to use indexes and clear WHERE conditions so the database can find rows quickly.

Maintainability also matters in team work. A clear block makes it easier for another student or developer to review the logic. It also makes it easier to fix bugs later. In projects, teachers often give marks for clean structure, correct naming, and safe handling, not only for “working output.”

3.5.1 Debugging and Testing

Testing means running your block with normal data and with edge cases. Normal data is the expected input, like a positive fee amount. Edge cases include zero, negative values, missing rows, and duplicate rows. A good block should behave in a predictable way in every case. During learning, you can print key values with DBMS_OUTPUT.PUT_LINE to confirm your path through IF statements and loops. You can also test your exception handlers by using an ID that does not exist to trigger NO_DATA_FOUND, or by using a query that matches multiple rows to trigger TOO_MANY_ROWS. When you do this on purpose, you learn faster and you fear errors less.

4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Block Skeleton

This is the basic skeleton. It shows the order of sections.

```
DECLARE
 -- declarations go here
 BEGIN
 -- executable statements go here
 EXCEPTION
 -- exception handlers go here
 END;
 /
```

In practice, you may omit DECLARE or EXCEPTION when you do not need them, but BEGIN and END must stay.

4.2 Example: Simple Output

This block prints a message for learning.

```
BEGIN
 DBMS_OUTPUT.PUT_LINE('Block executed successfully.');
 END;
 /
```


This shows a minimal block without DECLARE and without EXCEPTION.

4.3 Example: Variable and Assignment

This block declares a variable and uses := to assign it.

```
DECLARE
 v_count NUMBER := 5;
 BEGIN
 v_count := v_count + 1;
 DBMS_OUTPUT.PUT_LINE('Count is ' || v_count);
 END;
 /
```

This example also shows string joining with ||.

4.4 Example: SELECT INTO and Two Common Exceptions

This block reads one student name. It handles both “no row” and “many rows.”

```
DECLARE
 v_name student.student_name%TYPE;
 BEGIN
 SELECT student_name INTO v_name
 FROM student
 WHERE student_id = 101;

DBMS_OUTPUT.PUT_LINE('Student name: ' || v_name);
 EXCEPTION
 WHEN NO_DATA_FOUND THEN
 DBMS_OUTPUT.PUT_LINE('No student found for ID 101.');
 WHEN TOO_MANY_ROWS THEN
 DBMS_OUTPUT.PUT_LINE('More than one student matched the ID.');
 END;
 /
```

This teaches a key exam point: SELECT INTO expects exactly one row.

4.5 Example: IF and Loop in One Block

This block checks a rule and then repeats work.


```
DECLARE
 v_fee NUMBER := 1200;
 v_i NUMBER := 1;
 BEGIN
 IF v_fee <= 0 THEN
 DBMS_OUTPUT.PUT_LINE('Fee must be positive.');
 ELSE
 DBMS_OUTPUT.PUT_LINE('Fee is valid.');
 END IF;

WHILE v_i <= 3 LOOP
 DBMS_OUTPUT.PUT_LINE('Practice run: ' || v_i);
 v_i := v_i + 1;
 END LOOP;
 END;
 /
```

This shows decision and repetition in the executable part.

4.6 Example: Cursor Loop for Many Rows

This block reads many rows and prints a short output.

```
DECLARE
 CURSOR c_students IS
 SELECT student_id FROM student WHERE ROWNUM <= 5;
 BEGIN
 FOR r IN c_students LOOP
 DBMS_OUTPUT.PUT_LINE('Student ID: ' || r.student_id);
 END LOOP;
 END;
 /
```

A cursor helps process more than one row. It is common in batch tasks.

4.7 Case Study: Fee Payment with Validation and Logging

A payment rule should reject negative amounts. It should store valid payments. It should also provide a helpful message when something fails.

```
DECLARE
 v_student_id NUMBER := 101;
 v_amount NUMBER := 1500;
 BEGIN
 IF v_amount <= 0 THEN
 RAISE_APPLICATION_ERROR(-20001, 'Invalid payment amount.');
 END IF;

INSERT INTO payment(student_id, amount, paid_on)
 VALUES (v_student_id, v_amount, SYSDATE);

DBMS_OUTPUT.PUT_LINE('Payment stored for student ' || v_student_id);
 EXCEPTION
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Payment failed: ' || SQLERRM);
 END;
 /
```

In a real system, the exception handler can also write the error to an error_log table for later review.

4.8 Case Study: Library Fine Calculation

A library fine can depend on days late. This task needs a lookup and a calculation.

```
DECLARE
 v_book_id NUMBER := 5001;
```

```
 v_due_date DATE;
 v_return_date DATE := SYSDATE;
 v_days_late NUMBER;
 v_fine NUMBER;
 BEGIN
 SELECT due_date INTO v_due_date
 FROM library_issue
 WHERE book_id = v_book_id;

v_days_late := v_return_date - v_due_date;

IF v_days_late <= 0 THEN
 v_fine := 0;
 ELSE
 v_fine := v_days_late * 2;
 END IF;

DBMS_OUTPUT.PUT_LINE('Fine is ' || v_fine);
 EXCEPTION
 WHEN NO_DATA_FOUND THEN
 DBMS_OUTPUT.PUT_LINE('No issue record found for this book.');
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Fine task failed: ' || SQLERRM);
 END;
 /
```


This case shows how data lookup and safe handling work together.

4.9 Case Study: Bank Transfer with Safe Stop

A transfer needs two updates. If any step fails, the task should not leave accounts half updated.

```
DECLARE
 v_from_acct NUMBER := 10;
 v_to_acct NUMBER := 20;
 v_amount NUMBER := 500;
 v_balance NUMBER;
 BEGIN
 SELECT balance INTO v_balance
 FROM account
 WHERE acct_id = v_from_acct
 FOR UPDATE;
```

```
IF v_balance < v_amount THEN
 RAISE_APPLICATION_ERROR(-20002, 'Insufficient balance.');
 END IF;

UPDATE account
 SET balance = balance - v_amount
 WHERE acct_id = v_from_acct;

UPDATE account
 SET balance = balance + v_amount
 WHERE acct_id = v_to_acct;

DBMS_OUTPUT.PUT_LINE('Transfer completed.');
 EXCEPTION
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Transfer stopped: ' || SQLERRM);
 ROLLBACK;
 END;
 /
```

This shows a key idea: when a multi-step update fails, rollback may be needed to keep data consistent.

4.10 Case Study: Grade Processing in a Batch

A result system may update many rows. This example shows a clear loop and update.

```
DECLARE
 CURSOR c_results IS
 SELECT student_id, marks FROM result_table;
 v_grade VARCHAR2(2);
 BEGIN
 FOR r IN c_results LOOP
 IF r.marks >= 75 THEN
 v_grade := 'A';
 ELSIF r.marks >= 60 THEN
 v_grade := 'B';
 ELSIF r.marks >= 40 THEN
 v_grade := 'C';
 ELSE
 v_grade := 'F';
 END IF;
```

```
UPDATE result_table
 SET grade = v_grade
 WHERE student_id = r.student_id;
 END LOOP;

DBMS_OUTPUT.PUT_LINE('Grades updated.');
 EXCEPTION
 WHEN OTHERS THEN
 DBMS_OUTPUT.PUT_LINE('Grade update failed: ' || SQLERRM);
 END;
 /
```

This block is easy to grade because it is structured, indented, and clear.

4.10.1 Case Study: Attendance Warning Rule

A college system may warn a student when attendance is low. The task can be: read attendance percent, decide a warning level, and store a message. This is a good example because it uses selection, decision, and update.


```
DECLARE
 v_student_id NUMBER := 101;
 v_attend NUMBER;
 v_msg VARCHAR2(100);
 BEGIN
 SELECT attendance_percent INTO v_attend
 FROM attendance
 WHERE student_id = v_student_id;

IF v_attend < 50 THEN
 v_msg := 'Critical: meet the mentor.';
 ELSIF v_attend < 75 THEN
 v_msg := 'Warning: attendance is low.';
 ELSE
 v_msg := 'OK: attendance is sufficient.';
 END IF;

UPDATE attendance
 SET warning_message = v_msg
 WHERE student_id = v_student_id;

DBMS_OUTPUT.PUT_LINE(v_msg);
 EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No attendance row for this student.');
END;
/
```

This case study shows how a block can turn a numeric value into a clear policy message, which is useful for dashboards and student portals.

4.11 Study Tips for Online Learners

A simple practice plan is to memorize the block skeleton first. Then add one variable and one output line. Next, add one SELECT INTO. After that, add one IF condition. Then add one loop. Finally, add one exception handler. Run the block many times and change one line each time. Also test edge cases, such as missing rows, zero values, and duplicate rows. This habit builds both speed and accuracy for exams.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch Vocabulary Meaning (Humanized)
Block	A small PL/SQL program that runs as one complete unit from start to end
DECLARE	The part of a block where variables, constants, and cursors are defined
BEGIN	The part of a block where the actual program actions are written
EXCEPTION	The part of a block that handles errors in a safe and controlled way
END	The keyword that clearly marks the end of a PL/SQL block
Anonymous block	A temporary block that runs once and is not saved in the database
Procedure	A named block stored in the database that performs a specific task
Function	A named block stored in the database that always returns a value
Trigger	A stored block that runs automatically when a database event occurs
Package	A container that groups related procedures, functions, and variables
Variable	A named storage location used to hold values while the program runs
Scope	The part of the program where a variable name can be used
Lifetime	The time period during which a variable exists in memory
%TYPE	A feature used to copy a column’s data type safely
Cursor	A tool used to process multiple rows one row at a time

SELECT INTO	A SQL statement that places query results into variables
NO_DATA_FOUND	An error raised when a SELECT INTO query returns no row
TOO_MANY_ROWS	An error raised when a SELECT INTO query returns more than one row
SQLERRM	A system message that explains what error occurred
ROLLBACK	A command that cancels changes made during a transaction